# PSTAT 132 — Homework 1

Aldo Nunez

2026-10-09

```
library(DBI)
library(RSQLite)
library(dplyr)
library(dbplyr)
library(lubridate)
library(ggplot2)

PSTAT132db <- dbConnect(RSQLite::SQLite(), "PSTAT132-db.sqlite")
dbListTables(PSTAT132db)
```

```
## [1] "customers" "orders"    "products"
```

```
orders_tbl    <- tbl(PSTAT132db, "orders")
customers_tbl <- tbl(PSTAT132db, "customers")
```

## Question 1

Customers who placed orders in January 2023 and their total spend, ordered by total spend descending.

**SQL**

```
jan_start <- as.numeric(as.Date("2023-01-01"))
jan_end <- as.numeric(as.Date("2023-02-01"))

jan_spend_sql <- dbGetQuery(PSTAT132db, "
                            SELECT c.customer_id, c.name,
                            SUM(o.amount) AS total_spend
                            FROM orders o
                            INNER JOIN customers c ON o.customer_id = c.customer_id
                            WHERE o.order_date >= ? AND o.order_date < ?
                            GROUP BY c.customer_id, c.name
                            ORDER BY total_spend DESC;
                            ", params = list(jan_start, jan_end)) %>%
                            as_tibble()
jan_spend_sql
```

```
## # A tibble: 6 x 3
##   customer_id name  total_spend
##         <int> <chr>       <dbl>
## 1           1 Ava           235
## 2           4 Dee           200
## 3           2 Ben           180
## 4           6 Faye          160
## 5           5 Eli           150
```

```
## 6           3 Chi           115
```

**dbplyr**

```
jan_spend_dbp <- orders_tbl %>%
    filter(order_date >= !!jan_start,
           order_date < !!jan_end) %>%
    inner_join(customers_tbl, by = "customer_id") %>%
    group_by(customer_id, name) %>%
    summarise(total_spend = sum(amount, na.rm = TRUE), .groups = "drop") %>%
    arrange(desc(total_spend))

    show_query(jan_spend_dbp)
```

```
## <SQL>
## SELECT `customer_id`, `name`, SUM(`amount`) AS `total_spend`
## FROM (
##   SELECT `LHS`.*, `name`, `region`
##   FROM (
##     SELECT *
##     FROM `orders`
##     WHERE (`order_date` >= 19358.0) AND (`order_date` < 19389.0)
##   ) AS `LHS`
##   INNER JOIN `customers`
##     ON (`LHS`.`customer_id` = `customers`.`customer_id`)
## ) AS `q01`
## GROUP BY `customer_id`, `name`
## ORDER BY `total_spend` DESC
```

```
    jan_spend_dbp %>% collect()
```

```
## # A tibble: 6 x 3
##   customer_id name  total_spend
##         <dbl> <chr>       <dbl>
## 1           1 Ava           235
## 2           4 Dee           200
## 3           2 Ben           180
## 4           6 Faye          160
## 5           5 Eli           150
## 6           3 Chi           115
```

## Question 2

Number of orders per week in Q1 2023. Aggregation happens in the database: each order is assigned to the Monday that starts its week using SQLite date modifiers, and only the weekly counts are collected into R.

```
q1_start <- as.numeric(as.Date("2022-01-01"))
q1_end <- as.numeric(as.Date("2022-04-01"))

weekly_orders <- orders_tbl %>%
    filter(order_date >= !!q1_start,
           order_date < !!q1_end) %>%
    mutate(week_start = sql("DATE(order_date * 86400, 'unixepoch', '-6 days', 'weekday 1')")) %>%
    count(week_start, name = "n_orders") %>%
    arrange(week_start) %>%
    collect() %>%
```

```
    mutate(week_start = as.Date(week_start))

weekly_orders
```

```
## # A tibble: 0 x 2
## # i 2 variables: week_start <date>, n_orders <lgl>
```

```
ggplot(weekly_orders, aes(x = week_start, y = n_orders)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = n_orders), vjust = -0.4, size = 3) +
  scale_x_date(date_breaks = "1 week", date_labels = "%b %d") +
  labs(title = "Orders per Week, Q1 2023",
       x = "Week starting (Monday)", y = "Number of orders") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




## Question 3

`COLLATE NOCASE` changes how SQLite compares strings, so it is the natural choice for equality tests, `ORDER BY`, `GROUP BY`, or `DISTINCT` on a column where case should never matter, and it can be declared on the column itself so an index can use it. `LOWER(...)` instead transforms the value inside the query, which makes the intent explicit, is portable across database systems such as Postgres and MySQL, and is convenient for one-off filters like `LOWER(notes) LIKE '%gift%'`. The trade-off is that wrapping a column in `LOWER()` usually prevents the database from using an index, so it can be slower on large tables. In SQLite, both only fold ASCII letters by default, so neither is reliable for accented or non-English text.

## Question 4

**a) Top 3 customers by total spend**

```r
top3 <- orders_tbl %>%
  group_by(customer_id) %>%
  summarise(total_spend = sum(amount, na.rm = TRUE), .groups = "drop") %>%
  inner_join(customers_tbl %>% select(customer_id, name), by = "customer_id") %>%
  arrange(desc(total_spend)) %>%
  head(3)

top3 %>% collect()
```

```
## # A tibble: 3 x 3
##   customer_id total_spend name
##         <dbl>       <dbl> <chr>
## 1           1         235 Ava
## 2           4         200 Dee
## 3           2         180 Ben
```

**b) SQL generated by dbplyr**

```r
show_query(top3)
```

```
## <SQL>
## SELECT *
## FROM (
##   SELECT `LHS`.*, `name`
##   FROM (
##     SELECT `customer_id`, SUM(`amount`) AS `total_spend`
##     FROM `orders`
##     GROUP BY `customer_id`
##   ) AS `LHS`
##   INNER JOIN `customers`
##     ON (`LHS`.`customer_id` = `customers`.`customer_id`)
## ) AS `q01`
## ORDER BY `total_spend` DESC
## LIMIT 3
```

## Question 5

**a) Create an in-memory database and write order data to it**

```r
memdb <- dbConnect(RSQLite::SQLite(), ":memory:")

orders_df <- dbReadTable(PSTAT132db, "orders")
dbWriteTable(memdb, "orders", orders_df, overwrite = TRUE)
dbListTables(memdb)
```

```
## [1] "orders"
```

**b) Database-backed pipeline**

```r
customer_pipeline <- tbl(memdb, "orders") %>%
  group_by(customer_id) %>%
  summarise(
    total_spend = sum(amount, na.rm = TRUE),
    n_orders    = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(total_spend))
```

```r
show_query(customer_pipeline)
```

```
## <SQL>
## SELECT `customer_id`, SUM(`amount`) AS `total_spend`, COUNT(*) AS `n_orders`
## FROM `orders`
## GROUP BY `customer_id`
## ORDER BY `total_spend` DESC
```

```r
customer_summary <- collect(customer_pipeline)
class(customer_summary)
```

```
## [1] "tbl_df"     "tbl"        "data.frame"
```

```r
customer_summary
```

```
## # A tibble: 6 x 3
##   customer_id total_spend n_orders
##         <dbl>       <dbl>    <int>
## 1           1         235        3
## 2           4         200        1
## 3           2         180        3
## 4           6         160        1
## 5           5         150        2
## 6           3         115        2
```